Отчёт по программированию

Служба доставки беспилотниками

Вариант № 38

ИДЗ3

Орлов Фёдор Алексеевич

БПИ253

Москва, 2026

Содержание

1 Постановка задачи 2

2 Архитектура программы 2

3 Сценарий моделирования 3

4 Алгоритмы моделирования 3

4.1 Поиск маршрута 3

4.2 Назначение заказов 4

4.3 Доставка, зарядка и погода 4

5 Организация процессов и обмен данными 4

5.1 Дочерние процессы и неименованные семафоры 4

5.2 Дочерние процессы и именованные семафоры 5

5.3 Независимые процессы и разделяемая память 5

5.4 Хранение состояния в общей памяти 5

6 Консольный интерфейс и запуск 5

7 Завершение работы программы 6

8 Тестирование и результаты 6

9 Заключение 7

1. Постановка задачи

Цель работы — разработать консольное приложение, моделирующее работу службы доставки с помощью беспилотников. Программа предназначена для запуска в Linux и реализована на языке C с использованием системных вызовов и средств межпроцессного взаимодействия POSIX.

В модели предусмотрены склад, заказы, беспилотники, зарядные станции, прямоугольная карта с препятствиями и погодные условия. Диспетчер распределяет заказы между подходящими аппаратами. При выполнении доставки учитываются грузоподъёмность, запас энергии, расстояние до получателя и установленный срок.

Работа симулятора организована по модельным тактам. Основные действия выполняются отдельными процессами, которые согласуют доступ к общим данным с помощью семафоров. Для наблюдения за выполнением программы предусмотрен вывод событий в консоль и итоговой статистики.

2. Архитектура программы

Исходный код разделён на модули, каждый из которых отвечает за отдельную часть модели.

Модуль	**Назначение**
`map`	Хранение карты, препятствий и поиск маршрутов.
`plane`	Параметры беспилотников, запас энергии и перемещение.
`order`	Хранение заказов и изменение их состояний.
`station`	Зарядка аппаратов с учётом числа свободных мест.
`dispatcher`	Назначение заказов подходящим беспилотникам.
`weather`	Обновление погоды и управление возможностью полёта.
`simulation`	Модельное время, этапы доставки, события и статистика.
`scenario`	Загрузка и проверка входных данных.
`process_controller`	Координация процессов и последовательности модельных тактов.
`process_worker`	Выполнение действий одной группы объектов.
`process_ipc`	Создание общей памяти и семафоров, обмен данными.
`process_snapshot`	Представление состояния модели для передачи между процессами.

Главный процесс отвечает за запуск моделирования, обновление

времени, последовательность выполнения этапов и завершение работы. Действия с заказами, назначение беспилотников, перемещение и зарядка выполняются рабочими процессами.

3. Сценарий моделирования

В начале программа считывает сценарий, создаёт карту, добавляет склад, препятствия, станции, беспилотники и заказы. Каждый заказ имеет время поступления, пункт назначения, массу и срок доставки. Для беспилотника задаются координаты, грузоподъёмность, максимальный и начальный заряд, а также расход энергии.

Один модельный такт включает четыре этапа:

1. **Обработка заказов.** Проверяются поступившие заказы и сроки их выполнения.
2. **Работа диспетчера.** Свободным беспилотникам назначаются подходящие заказы.
3. **Движение беспилотников.** Аппараты следуют к складу, забирают груз, доставляют его получателю или направляются к зарядной станции.
4. **Работа зарядных станций.** Восстанавливается энергия аппаратов, находящихся на зарядке, и обслуживается ожидание свободного места.

Порядок этапов обеспечивает последовательное изменение состояния в пределах одного такта. При этом каждый рабочий процесс самостоятельно ожидает разрешения на выполнение своей части работы. Такой подход позволяет согласованно моделировать взаимодействие нескольких участников системы.

4. Алгоритмы моделирования

4.1. Поиск маршрута

Карта представлена двумерной сеткой. Из каждой свободной клетки возможен переход в соседнюю свободную клетку по горизонтали или вертикали. Для поиска кратчайшего маршрута используется обход в ширину (BFS).

Алгоритм начинает обход из исходной клетки, последовательно посещает соседние доступные клетки и сохраняет предков для восстановления маршрута. При размере карты $R \times C$ время работы и дополнительная память составляют $O(RC)$.

4.2. Назначение заказов

Диспетчер рассматривает доступные заказы и свободные беспилотники. Для назначения проверяются грузоподъёмность аппарата, наличие маршрутов и достаточный запас энергии для движения к складу, получателю и зарядной станции. Также учитывается возможность завершить доставку до установленного срока.

Из подходящих аппаратов выбирается беспилотник с наименьшей расчётной длительностью доставки. После назначения он переходит к выполнению заказа.

4.3. Доставка, зарядка и погода

Беспилотник последовательно проходит несколько состояний: движение к складу, получение груза, движение к получателю и возвращение на зарядную станцию. На каждом такте аппарат перемещается на одну клетку, после чего обновляется его энергия.

Зарядная станция имеет ограниченное количество мест. При наличии свободного места аппарат начинает зарядку; остальные ожидают обслуживания. Заряд восстанавливается с заданной скоростью.

В модели предусмотрены благоприятная и неблагоприятная погода. Во время неблагоприятной погоды полёты приостанавливаются, а зарядные станции продолжают работу. После улучшения погоды движение возобновляется. По завершении заказ получает соответствующий итоговый статус.

5. Организация процессов и обмен данными

Для моделирования реализованы три способа организации взаимодействующих процессов. Все они используют одну предметную модель доставки.

5.1. Дочерние процессы и неименованные семафоры

В первом варианте управляющий процесс создаёт четыре дочерних процесса с помощью `fork()`. Для обмена состоянием используется общая область памяти, выделенная через `mmap()` с флагом `MAP_SHARED`. Синхронизация выполняется неименованными POSIX-семафорами, инициализированными функцией `sem_init()` с параметром `pshared=1`.

Каждому процессу назначена своя роль. Рабочий процесс ожидает разрешения начать очередной этап и после его выполнения уведомляет контроллер.

5.2. Дочерние процессы и именованные семафоры

Во втором варианте дочерние процессы также создаются через `fork()`, но для синхронизации применяются именованные POSIX-семафоры. Они создаются с помощью `sem_open()` и используются для запуска этапов, подтверждения их окончания и взаимного исключения. Общее состояние размещается в POSIX shared memory.

5.3. Независимые процессы и разделяемая память

В третьем варианте контроллер и рабочие процессы запускаются как самостоятельные программы. Все участники подключаются к одному пространству разделяемой памяти с помощью `shm_open()` и `mmap()`. Для согласования действий используются именованные семафоры.

Контроллер ожидает подключения четырёх работников: `orders`, `dispatcher`, `drones` и `stations`. Для каждого этапа он разрешает работу соответствующему процессу и ожидает подтверждения завершения. Благодаря этому все участники работают с согласованным состоянием модели.

5.4. Хранение состояния в общей памяти

Состояние симулятора содержит как обычные поля, так и динамические структуры. Для передачи данных между процессами используется специальное представление состояния без адресных указателей.

Модуль `process_snapshot.c` сохраняет необходимые параметры заказов, беспилотников, станций и текущих этапов доставки. Рабочий процесс восстанавливает данные в локальных структурах, выполняет действия своей роли и записывает обновлённое состояние в общую память. Доступ к состоянию защищён семафором взаимного исключения.

6. Консольный интерфейс и запуск

Программа собирается в Linux с помощью `Makefile`. Для сборки всех исполняемых файлов используется команда:

```
make
```

Примеры запуска:

```
./fly_fork_unnamed --scenario scenarios/demo.txt
./fly_fork_named --scenario scenarios/demo.txt --delay 50
bash scripts/run_external.sh --scenario scenarios/demo.txt --delay 50
```

Третий способ допускает ручной запуск контроллера и рабочих процессов в отдельных терминалах. Все программы используют одинаковый идентификатор экземпляра `--instance`:

```
./fly_external_shm --instance demo --scenario scenarios/demo.txt
./fly_process_worker --transport shm --role orders --instance demo --
   scenario scenarios/demo.txt
./fly_process_worker --transport shm --role dispatcher --instance
   demo --scenario scenarios/demo.txt
./fly_process_worker --transport shm --role drones --instance demo --
   scenario scenarios/demo.txt
./fly_process_worker --transport shm --role stations --instance demo
   --scenario scenarios/demo.txt
```

При работе в консоль выводятся ключевые события: поступление заказа, назначение аппарата, получение груза, перемещение, доставка, изменение погоды, прибытие на станцию и зарядка. В конце программа показывает количество доставленных и невыполненных заказов, пройденное расстояние и долю успешных доставок.

Параметр `--delay` задаёт задержку между тактами для наглядного наблюдения. С помощью `--scenario` можно выбрать входной файл, а `--quiet` сокращает количество промежуточных сообщений. Запуск независимых процессов также автоматизирован скриптом `scripts/run_external.sh`.

7. Завершение работы программы

Симуляция завершается после обработки всех заказов и активных маршрутов либо по достижении заданного ограничения модельного времени. Предусмотрено также завершение по сигналам `SIGINT` и `SIGTERM`.

При остановке контроллер уведомляет рабочие процессы о завершении, после чего освобождаются ресурсы межпроцессного взаимодействия. Для дочерних процессов используется `waitpid()`. Общая память отключается с помощью `munmap()`, именованные объекты удаляются через `sem_unlink()` и `shm_unlink()`. Это позволяет корректно завершать работу и повторно запускать моделирование.

8. Тестирование и результаты

Для проверки программы используются модульные и интеграционные тесты. Модульные тесты охватывают маршрутизацию, расход энергии, обработку заказов, работу диспетчера, зарядку, погодные условия и чтение сценариев.

Интеграционные тесты запускают все три процессные реализации на нескольких входных сценариях, сравнивают итоговую статистику и проверяют завершение по сигналу прерывания. Общий набор тестов запускается командой:

```
make test
```

Выполнено 10 модульных и 15 интеграционных проверок. Все тесты завершились успешно. При запуске различных вариантов программы получены согласованные результаты моделирования.

Таблица 1: Результаты тестовых сценариев

Сценарий	**Такты**	**Заказы**	**Доставлено**	**Отказано**	**Шаги**
`small.txt`	8	1	1	0	8
`demo.txt`	150	6	4	2	163

Дополнительно проверены сценарии с ограниченными сроками доставки и большим числом заказов. Отдельно проверялась остановка процесса пользователем. Таким образом, тестирование подтверждает согласованную работу модели и механизмов взаимодействия процессов.

9. Заключение

В ходе работы разработано консольное приложение на языке C для моделирования службы доставки беспилотниками в Linux. Реализованы обработка заказов, поиск маршрутов, управление запасом энергии, зарядные станции, погодные условия и сбор итоговой статистики.

Для организации моделирования использованы взаимодействующие процессы и средства POSIX: `fork()`, разделяемая память и семафоры. Подготовлены варианты с дочерними и независимо запускаемыми процессами. Работа программы сопровождается информативным выводом, а для проверки предусмотрены автоматические тесты и набор сценариев. Полученная реализация демонстрирует согласованную работу процессов при выполнении общей задачи моделирования.